Week 9: Flask and SQLite: CRUD

One connection per request

```
import sqlite3
from flask import g

def get_db():
    if "db" not in g:
        g.db = sqlite3.connect(app.config["DATABASE"])
        g.db.row_factory = sqlite3.Row
        g.db.execute("PRAGMA foreign_keys = ON")
    return g.db

@app.teardown_appcontext
def close_db(exception=None):
    db = g.pop("db", None)
    if db is not None:
        db.close()
```

`g` lasts for one request. `Row` lets you write `row['name']` and `{{ row.name }}`. Foreign keys must be switched on for **every** connection.

CRUD

	Method	SQL
Create	POST	`INSERT INTO t (a, b) VALUES (?, ?)`
Read	GET	`SELECT ... WHERE id = ?`
Update	POST	`UPDATE t SET a = ? WHERE id = ?`
Delete	POST	`DELETE FROM t WHERE id = ?`

HTML forms only send GET or POST, so update and delete use POST.

Read

```
rows = get_db().execute(
    "SELECT * FROM students WHERE name LIKE ?",
    (f"%{q}%",)).fetchall()
row = db.execute(
    "SELECT * FROM students WHERE student_id = ?",
    (sid,)).fetchone()
if row is None:
    abort(404)
```

Create, update, delete: with db:

```
db = get_db()
with db:           # commit, or roll back on error
    cur = db.execute(
        "INSERT INTO students (name, class) "
        "VALUES (?, ?)",
        (name, cls))
new_id = cur.lastrowid      # the new primary key
flash("Added.")
return redirect(url_for("student", student_id=new_id))
```

- `cur.lastrowid` = new id. `cur.rowcount` = rows changed (0 → 404?).
- Store an empty optional field as **NULL**: `email or None`.
- Catch duplicates: `try: with db: ... except sqlite3.IntegrityError:` and show a friendly message.

Delete in the right order

```
with db:       # all three, or none
    db.execute("DELETE FROM scores "
               "WHERE student_id=?", (i,))
    db.execute("DELETE FROM student_cca "
               "WHERE student_id=?", (i,))
    db.execute("DELETE FROM students "
               "WHERE student_id=?", (i,))
```

Children first, then the parent. With foreign keys on, deleting the parent first fails (`FOREIGN KEY constraint failed`): good. With them off, you get silent orphan rows: bad.

Upsert (insert or update)

```
INSERT INTO scores (student_id, subject_id, score)
VALUES (?, ?, ?)
ON CONFLICT (student_id, subject_id)
DO UPDATE SET score = excluded.score;
```

Sorting safely

```
SORTABLE = {"name": "s.name", "class": "s.class"}
col = SORTABLE.get(request.args.get("sort"), "s.name")
db.execute(f"SELECT ... ORDER BY {col}")
#                 col is OUR text, not the user's
```

> A column name cannot be a `?`. Use a **whitelist** dictionary: the user chooses a key, and the SQL contains only your own text.

Working copy of the database

The demo copies `school.db` to `instance/school.db`. Delete the `instance/` folder to start again. Look inside with the SQLite Viewer or `sqlite3 instance/school.db`.

Checklist for every route

- `?` placeholders only. No f-strings with user data.
- Changes are POST, inside `with db:`, then redirect.
- Validate first; unknown id → 404; handle duplicates.